

Security Rules Work: A Multi-Model Study of LLM Coding Agent Guardrails

Adhithya Rajasekaran · Axonome · adhithya@axonome.xyz · ORCID 0009-0004-1682-7958

SOUPS 2026
Poster Draft

Question

When developers write security rules for AI coding agents, does wording matter?

Prompt-engineering folklore says negative rules such as NEVER use eval() can backfire by making the forbidden API more salient. We tested whether this holds for LLM coding agents.

Short answer: rules help. Prohibition wording itself is not the main problem.

Study Design

6

models: 3
GPT/Codex
+ 3 Claude

6

security
prompts
across 4
CWE
classes

2,160

valid trials,
zero final
errors

Conditions

- Control: task prompt only.
- Negative rule: prohibit insecure pattern.
- Positive rule: suggest safer alternative.

Security classes

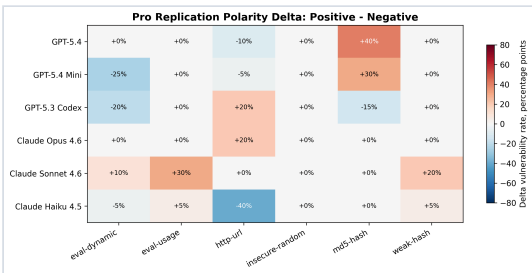
Main Result: Rules Reduce Vulnerable Code

Every model generated fewer vulnerable outputs when a relevant security rule was present.



Polarity Does Not Generalize

Positive framing is not consistently safer than prohibition framing. The direction changes by model and prompt.



Red: positive framing produced more vulnerable code.
Blue: positive framing produced less.

Takeaways

Aggregate Counts

Model	Control	Negative	Positive	Any rule p
GPT-5.4	66/120	18/120	24/120	8.41e-13
GPT-5.4 Mini	82/120	27/120	27/120	4.10e-17
GPT-5.3 Codex	81/120	15/120	12/120	1.52e-27
Claude Opus 4.6	58/120	0/120	4/120	3.35e-28

CWE-94 code execution, CWE-328 weak hashing, CWE-319 plaintext HTTP, CWE-338 insecure randomness.

Model	Control	Negative	Positive	Any rule p
Claude Sonnet 4.6	54/120	18/120	30/120	1.77e-06
Claude Haiku 4.5	104/120	31/120	24/120	3.26e-32

Add rules for known CWE classes.
Rule presence is the robust effect across all six models.

Do not rely on wording folklore.
Negative-vs-positive framing is not significant for any model in aggregate.

Measure against the target agent stack.
Model-specific behavior is large enough to change the best rule wording locally.

Ethics. No human subjects were studied. The experiment analyzes model-generated code under synthetic security prompts.

Artifacts. Dataset, runner, figures, and drafts are in the project repository.

Poster format. A4 landscape draft with high-contrast sans-serif text for SOUPS review submission.